Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Motivation

Architektur- und Entwurfsmuster der Softwaretechnik

Dokumenthistorie

Autor:	Prof. Dr. Bernd Hafenrichter
Version:	1.0
Letze Änderung:	16.03.2025

Motivation

Warum bleibt Softwareentwicklung immer interessant?

- Software unterliegt ständiger Veränderung
  - Funktionalität
  - Technologie Durch
  - Gesetzliche Bestimmungen
- Anpassungen sind notwendig damit die Software weiterhin eingesetzt werden kann.
- Sind die Anpassungen nicht einfach durchzuführen so altert die Software

Motivation

http://blog.axxg.de/indikatoren-fuer-schlechtes-design-softwarearchitektur/

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Starrheit (rigidity)
  - Ein starres Design löst genau das geforderte Problem
  - Ein starres Design ist sehr unflexibel gegenüber Anpassung und Veränderung

- Zerbrechlichkeit (fragility)
  - Änderungen an einer Stelle führen zu unvorhergesehenen Auswirkungen in anderen Systembestandteilen

Schnittstelle, Softwarestrukturen, Daten verwenden

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Unbeweglichkeit (immobility)
  - Fehlende Möglichkeit der Wiederverwendung einzelner Module anderer/bestehender Anwendungen
  - Hoher Aufwand für das herauslösen einzelner Module führt dazu dass die Wiederverwendung nicht genutzt wird

Email-Module programmieren

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Zähigkeit (viscosity)
  - Meist gibt es mehrere Varianten ein Problem zu lösen oder eine Erweiterung vorzunehmen. Davon sind einige designkonform und andere wiederum nicht. Diese nicht mit dem Design vereinbare Änderungen bezeichnet man als „Hacks“ oder „kludge“.
  - Wenn es also schwieriger ist, eine konforme Lösung zu implementieren, spricht man von einem zähen Design. Mehrere Hacks verschlechtern die Wartbarkeit und erhöhen die Komplexität der Software.

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Zähigkeit (viscosity)
  - Eine weitere Form der Zähigkeit tritt im Workflow der Entwicklung auf.
  - Dauert das Kompilieren der Software sehr lange, scheuen sich die Entwickler davor Änderungen vorzunehmen, die eine Neuübersetzung der Software zur Folge haben.
  - Einen ähnlichen Effekt bewirkt eine zu komplexe Versionsverwaltung oder eine schlechte IDE.

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Kompliziertheit
  - Der erste Indikator „Starrheit“ fordert vom Entwickler/Architekten weitblickend zu entwerfen/programmieren.
  - Übertreibt man diesen Ratschlag, ist der Aufbau der Software komplizierter als es die Komplexität der Aufgabe erfordert.
  - Dabei wird haufenweise zusätzliche Flexibilität eingebaut, um auf zukünftige Änderungen vorbereitet zu sein. Dies verkompliziert das Design, ohne zur Problemlösung beigetragen zu haben. Es muss also mehr Code gewartet werden als nötig.

Wenn der Code schwer ist und man nichts tun, um den Code zu verbessern.

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Unnötige Redundanz
  - Die selben Konzepte bzw. Anforderungen werden an unterschiedlichen Stellen im Design oder Entwurf redundant ausgearbeitet.
  - Die einfachste Form entsteht durch Copy- und Paste-Programmierung.
  - Die Folge ist, dass vorhandene Abstraktionen nicht erkannt und als solche herausgearbeitet werden. Somit muss eine Änderung an mehreren Stellen konsistent durchgeführt werden

Warum der Kontext

Motivation

Eigenschaften welche eine einfache Anpassung/Verwendung verhindern

- Unverständlichkeit
  - Die verschiedenen Einheiten und Ebenen der programmierten Software sind schwer zu durchschauen. Schlechte Namensgebung und ungünstige Strukturierung erschweren die Einarbeitung in die Software und damit die Überarbeitung / Erweiterung. Bestehende Konzepte und Konventionen werden vergessen oder weg gelassen.

Motivation

- Die dargestellten Eigenschaften sind die Folge eines schlechten Systementwurfs.
- Diese Eigenschaften gilt es zu vermeiden

- Systeme müssen so gebaut sein dass folgend Eigenschaften erfüllt sind:
  - korrekt (=Das System tut was es soll)
  - erweiterbar (=Das System ist auf Änderungen vorbereitet)
  - verständlich (=Das System erschließt sich dem Menschen)

Verwendung von Mustern und Best Practices welche Ihre Positive Wirkung bereits gezeigt haben.
Oder: Lernen von den Experten